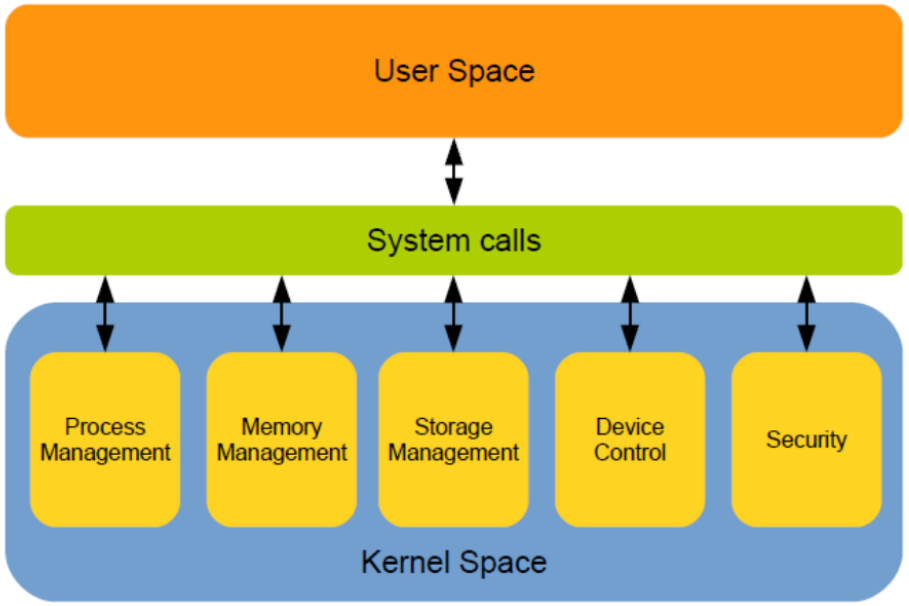


Main Memory

Ivy Peng
Associate Professor
EECS, KTH

OS Organization

- Basic organization of an Operating System
  - Process management
  - **Memory management**
  - Storage management and File system
  - Device control and I/O subsystem
  - Protection and security





Outline

- **Memory Hardware**
- Address Protection
- Address Binding
- Contiguous Memory Allocation
- Memory Fragmentation
- Advanced Memory Topics: Tiered, NUMA

By the End of This Lecture

- **Q1** Address binding has three stages: which stages, which needs the MMU?
- **Q2** Given memory holes and requests, apply first / best / worst fit and the trade-off?
- **Q3** External vs internal fragmentation: How does paging trade one for the other?

OS services in Memory Management

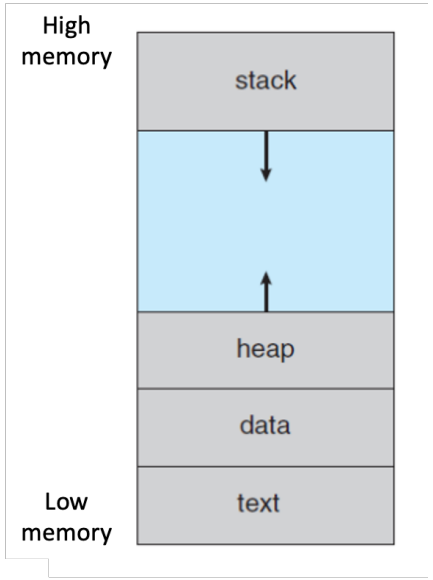
In order to efficiently execute a program:

- Load the program into memory -> e.g., memory allocation
- Determine data accessed in the program in the physical memory -> e.g., paging
- Load accessed data into memory -> e.g., address protection
- Orchestration among multiple processes -> e.g., swapping

A simple Copy program

```
1.  #define N 256
2.  double a[N];
3.  double b[N];

4.  int copy(){
5.          for(int i=0;i<N;i++)
6.                  b[i] = a[i];
7.  }
```



Check Memory Information in Linux

- *vmstat* can be useful for checking page faults, swap usage, and memory allocation

```
[bopeng@sleipner ~]$ vmstat 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 0  0      0 4111564   3752 11718728    0    0     0     6    2    1  0  0 100  0  0
 0  0      0 4111564   3752 11718728    0    0     0     0  200  204  0  0 100  0  0
 1  0      0 4111564   3752 11718728    0    0     0     0  223  217  0  0 100  0  0
```

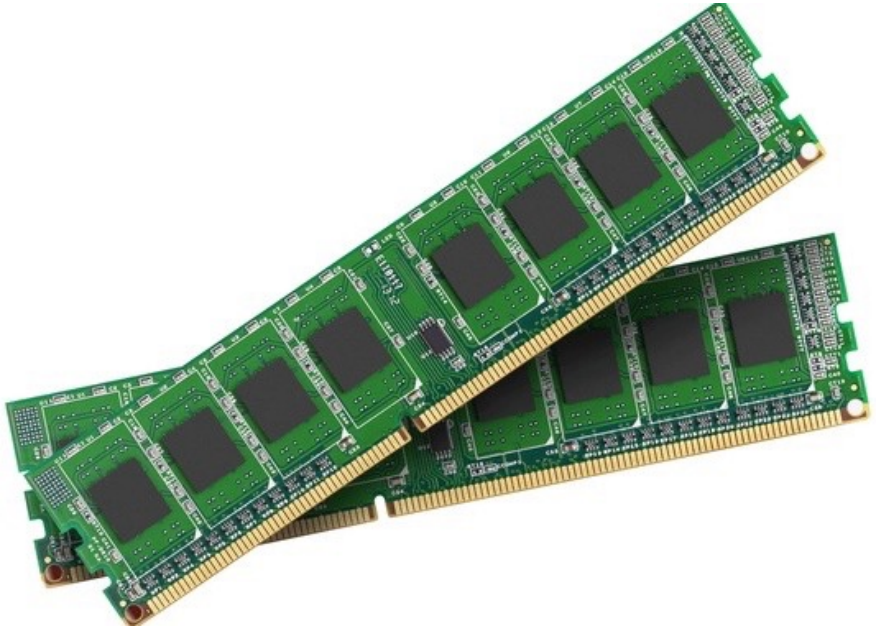
Question: can a new memory allocation request of 5GB be supported?

Why Memory Matters

- Data and Programs are stored in Memory
- Programs must be loaded into memory before execution
- Programs fetch the next instruction to execute from memory via Program Counter
- Programs access data stored in memory via load (ld) / store (st) instructions

Memory Hardware

- Double Data Rate (DDR) Memory is the most common memory technologies used for main memory of computing systems
  - Fast
  - Cheap
  - Volatile
- Memory is characterized by its capacity (GB), bandwidth (GB/s), and latency (ns)
  - Further distinction in read /write bandwidth
  - Sequential access vs. random access latency
- New memory technologies are becoming more common on Data Centers
  - High-bandwidth memory (HBM)
  - Non-volatile byte addressable memory



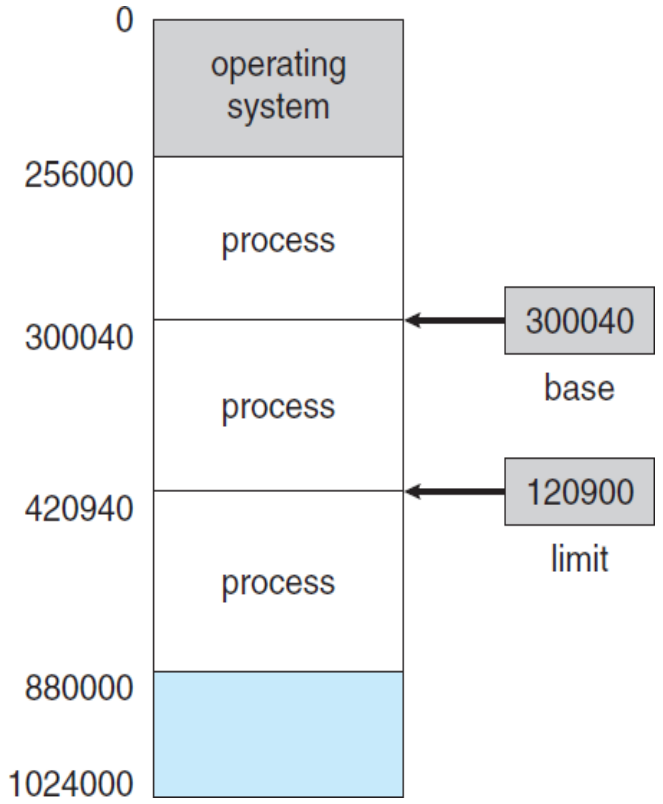


Outline

- Memory Hardware
- **Address Protection**
- Address Binding
- Contiguous Memory Allocation
- Memory Fragmentation
- Advanced Memory Topics: Tiered, NUMA

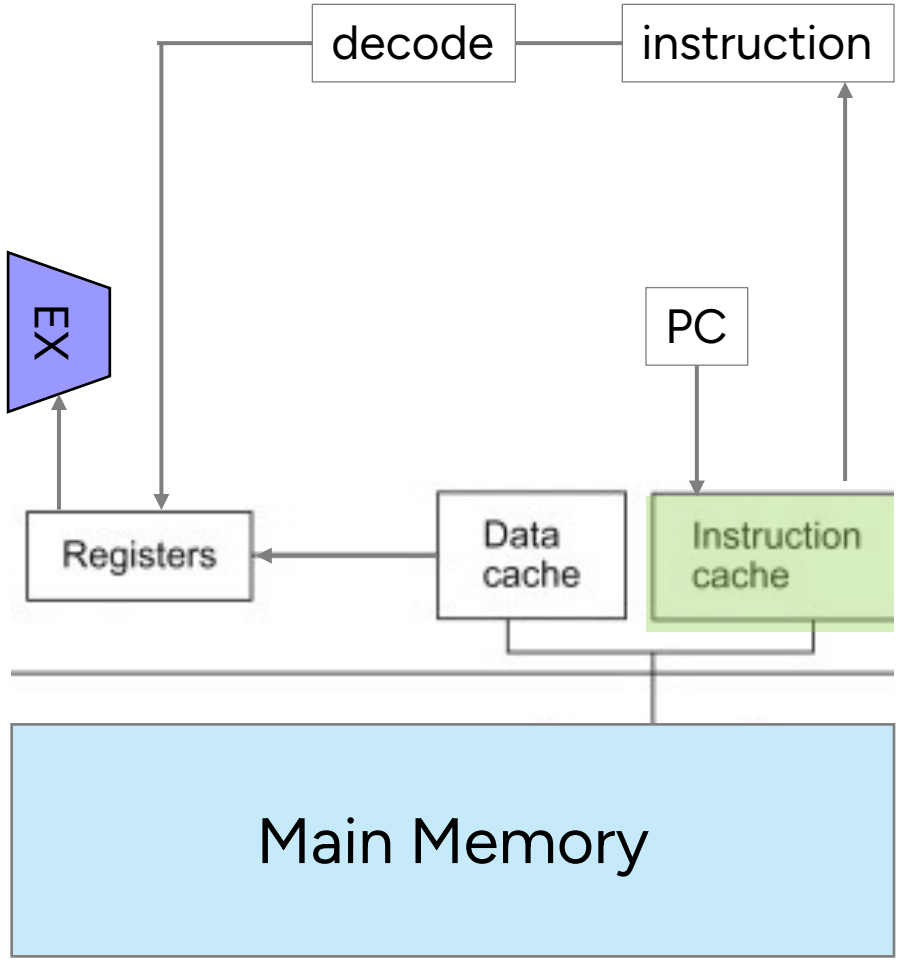
Address Protection

- Multiple **users** run multiple processes on a system
- The OS needs to isolate and protect **processes** from one another
- Address protection is provided by hardware: a separate memory space for each process
- CPU must check every memory access generated in user mode is accessing the legal logical addresses that a process can access



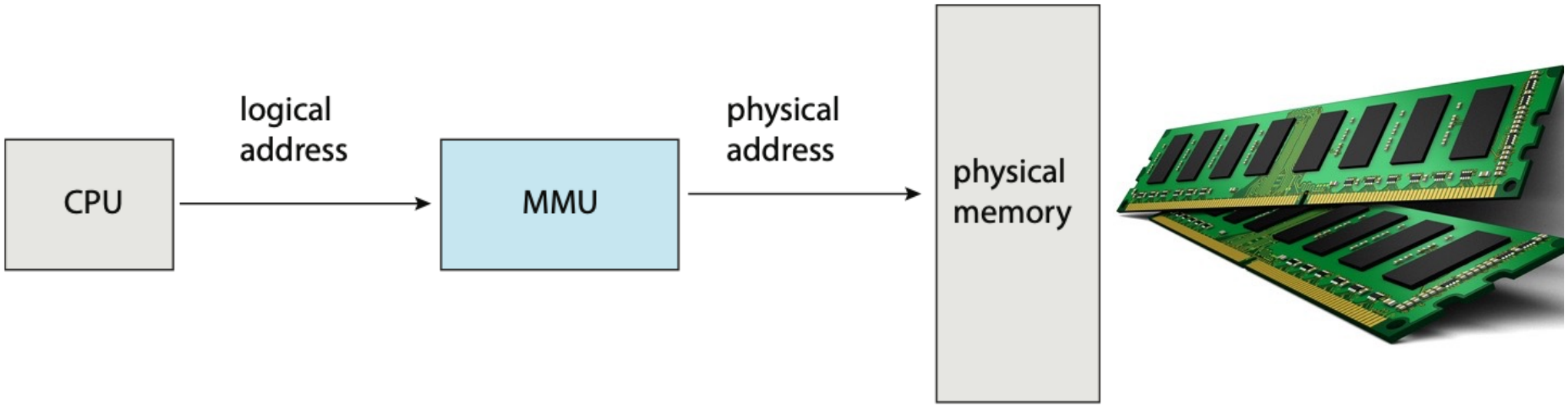
Recap: Execution Pipeline

- A program must be first loaded into memory and be executed by CPU to become a process
- The **CPU fetches instructions** from memory by program counter.
- Instructions may take **memory addresses** as arguments.
- An address used by the CPU is a **logical address**
- An address seen by the memory is a **physical address**



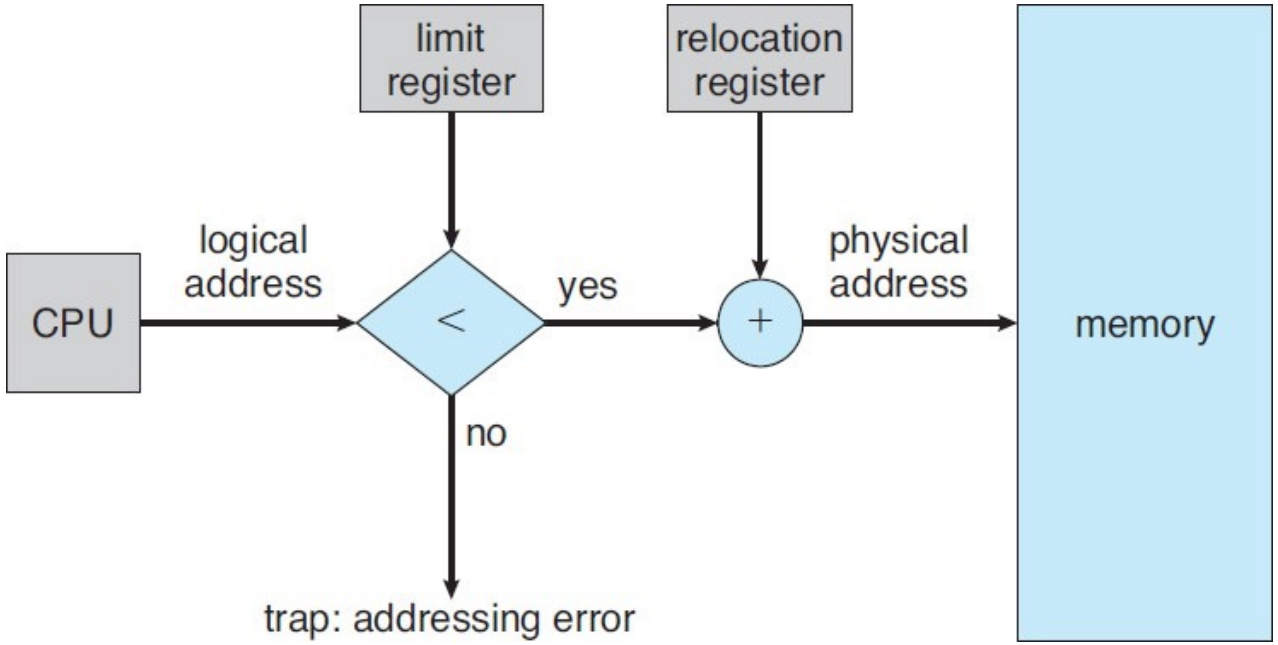
Logical vs. Physical Address Space

- We use logical address and virtual address interchangeably
- MMU is a special hardware unit that maps virtual to physical address at runtime



Memory Protection

- Relocation (interchangeably, base) registers are used to protect user processes from each other, and from changing OS code and data.
  - Base register contains value of smallest physical address.
  - Limit register contains range of logical addresses.
- Any attempt by a user program to access OS memory or other users' memory results in a trap to the OS, which treats the attempt as a fatal error.



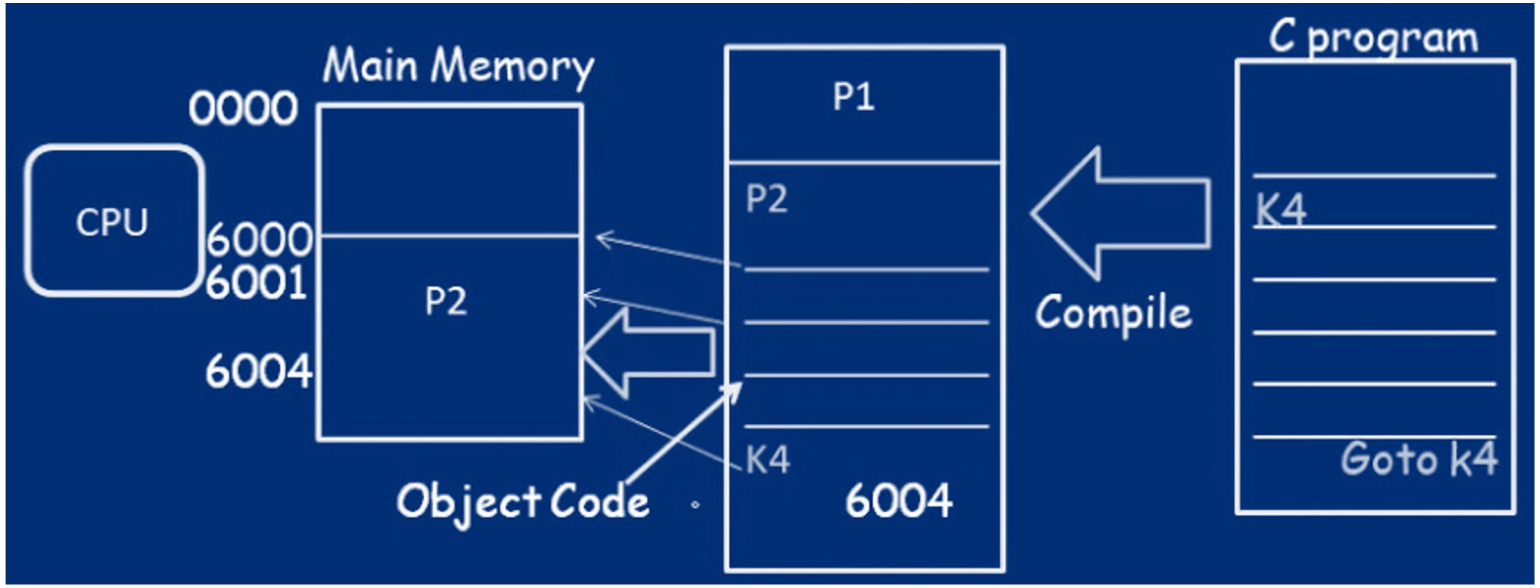


Outline

- Memory Hardware
- Address Protection
- **Address Binding**
- Contiguous Memory Allocation
- Memory Fragmentation
- Advanced Memory Topics: Tiered, NUMA

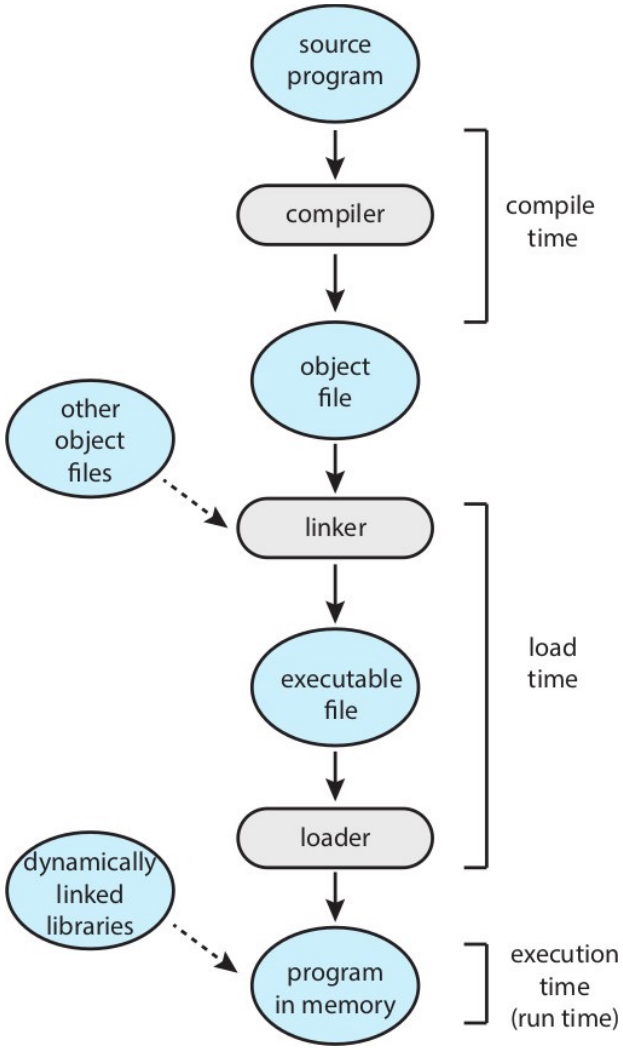
Address Binding

- A program is a binary executable file stored on disk**.** To run a program, it must be loaded into (physical) memory to be executed by CPU
- A user process can reside in any part of the physical memory
- Addresses are symbolic (e.g. K4) in program, but CPU only access addresses (e.g. 6004):
  - Need address binding



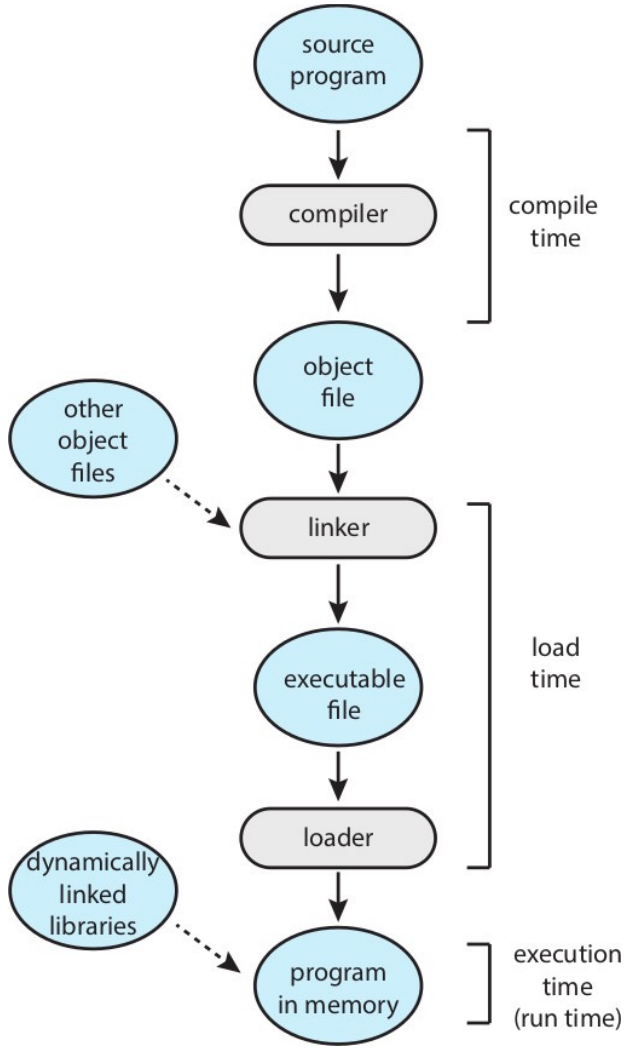
Address Binding

- Address binding is a process of mapping from one address space to another
  - Eventually from symbolic address in source program to absolute address in virtual memory
- Address Bindings could happen in 3 stages:
  - Compile time
  - Load time
  - Execution time



Compile Time

- Two options
- If the memory address for the process (say: 4000) is known at compilation time, absolute code is generated with '**hardcoded**' address
  - Need recompilation if the starting address of the process changes
- The most common option: generate **relocatable code,** resolve the absolute address until load time




```c
#include <stdlib.h>

int main(int argc, char** argv) {

  int var = 3;
  return 0;

}
```

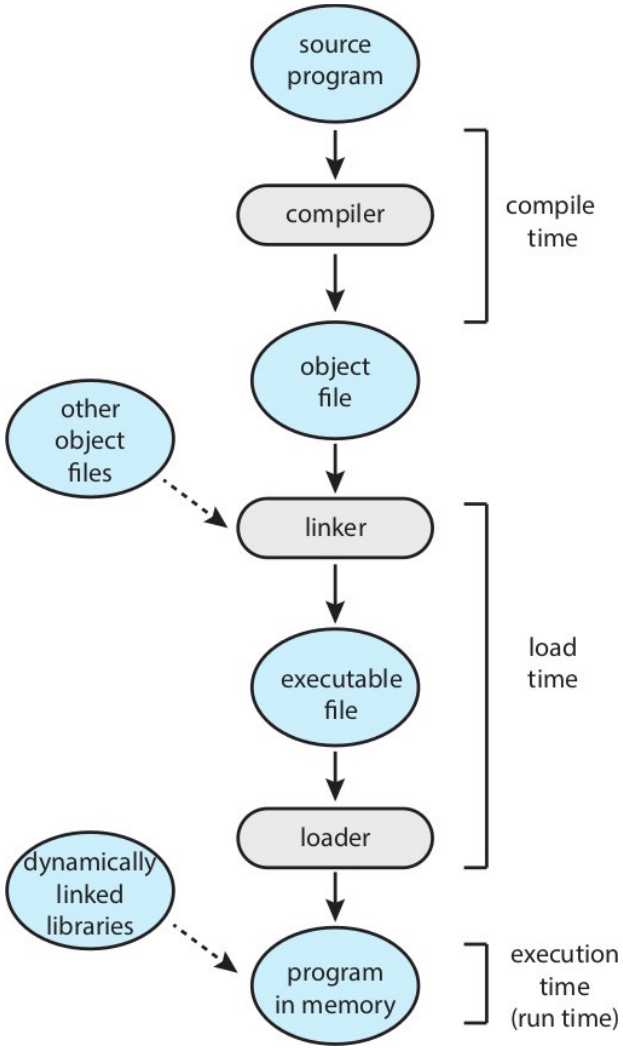
```
$ gcc -fPIC -c test.c
$ objdump -d test.o
```

```
Disassembly of section __TEXT,__text:

0000000000000000 <ltmp0>:
       0: ff 83 00 d1   sub     sp, sp, #32
       4: e8 03 00 aa   mov     x8, x0
       8: 00 00 80 52   mov     w0, #0
       c: ff 1f 00 b9   str     wzr, [sp, #28]
      10: e8 1b 00 b9   str     w8, [sp, #24]
      14: e1 0b 00 f9   str     x1, [sp, #16]
      18: 68 00 80 52   mov     w8, #3
      1c: e8 0f 00 b9   str     w8, [sp, #12]
      20: ff 83 00 91   add     sp, sp, #32
      24: c0 03 5f d6   ret
```

Load Time

- The compiler must generate relocatable code.
- If the starting address of the process changes, we only need to reload the user code to incorporate the changed address range.
- Unix loader's main tasks:
  - Check permission
  - Load the executable from the disk into main memory: mmap()
    - With the start of memory address, resolve the relative address into absolute memory address
  - initializing registers
  - Jump to the program entry point



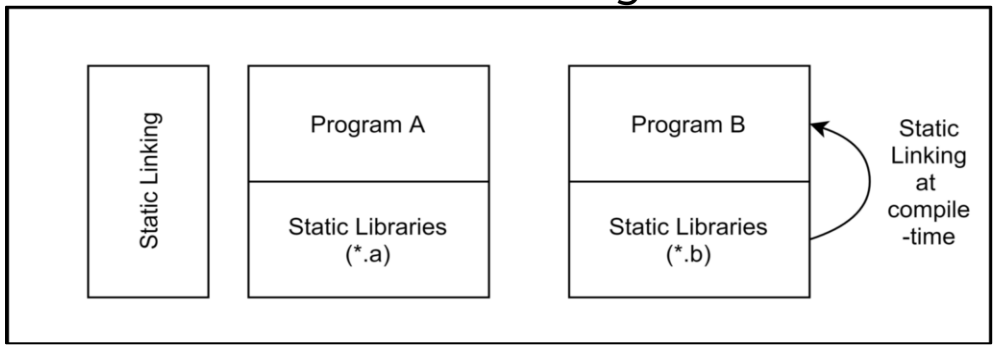
Linking and Loading

- Linking: connecting all the modules or the function of a program for program execution.
  - **Static linking**: System libraries and program code combined by the loader into the binary program image
    - If the library changes, need to recompile the program
  - **Dynamic linking**: Linking is postponed until execution time. Useful for shared libraries.

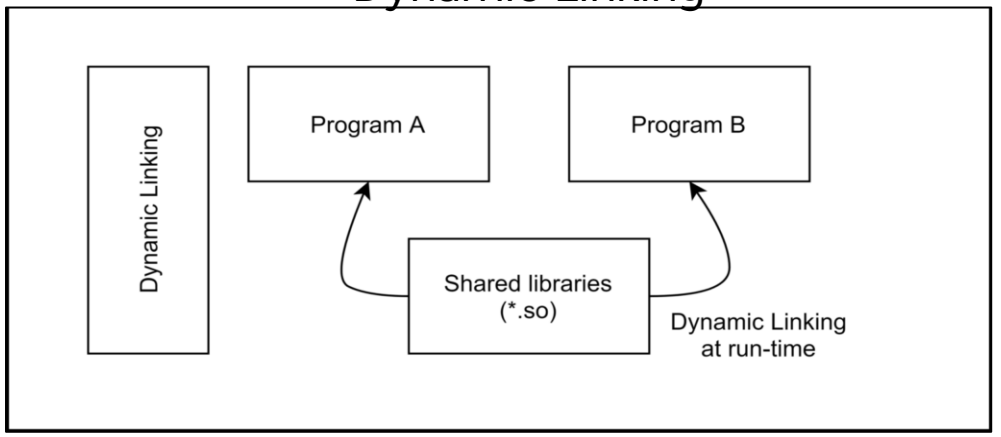
```
$ gcc test.c -static -labc -o test_static #libabc.a
$ gcc test.c -labc -o test_dynamic        #libabc.so
```

- Loading comes after linking: loading the program from disk to the main memory for execution.

Static Linking



Dynamic Linking

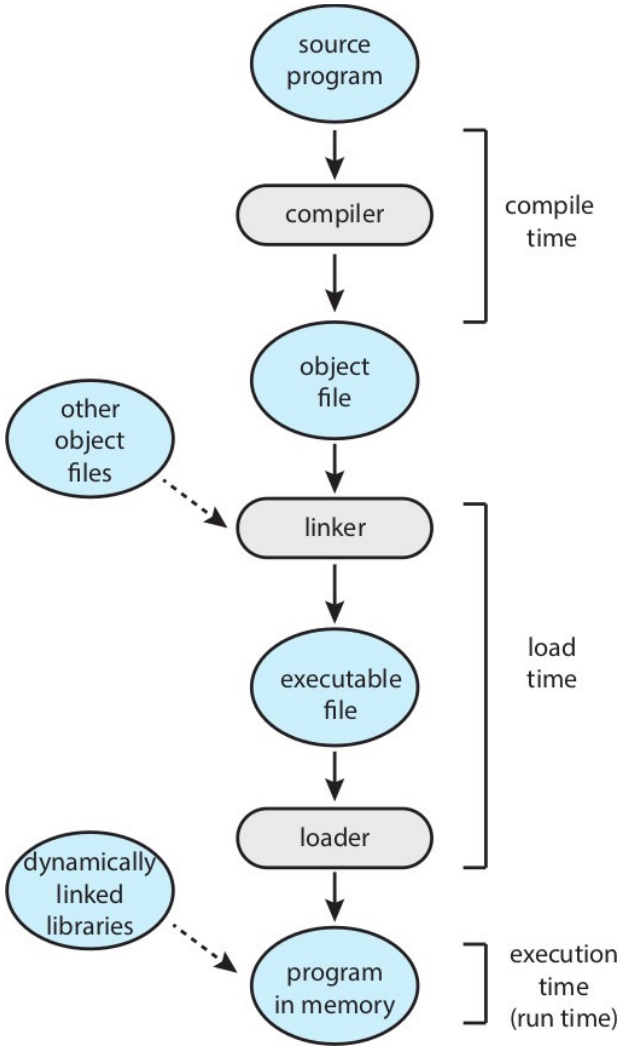


Dynamic Loading

- Routine/library is not loaded until it is called.
- The main program is loaded into memory and is executed.
- When a routine is needed, if it has not been loaded, the loader loads the routine into memory.
- Pros: Better memory-space utilization; unused routine is never loaded.
- Cons: ?

Execution Time

- If the process can be moved during its execution from one memory segment to another, binding delayed until runtime
- Need hardware support called the memory-management unit (MMU)
- Modern systems use execution-time binding.





Outline

- Memory Hardware
- Address Binding
- **Contiguous Memory Allocation**
- Memory Fragmentation
- Advanced Memory Topics: Tiered, NUMA

Contiguous Memory Allocation

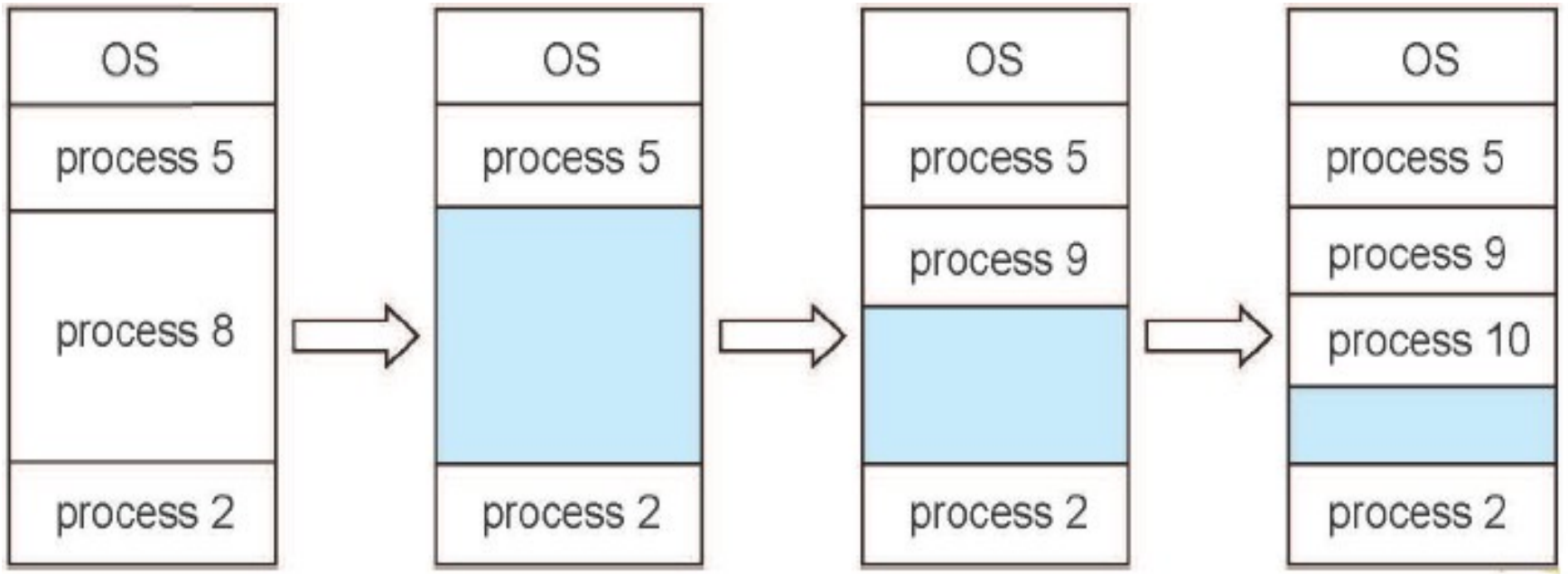
- Each process contained in single contiguous section of memory. i.e.: request X bytes memory -> is allocated to a contiguous address space [R, R+X]

- Main memory must support both OS and user processes. Main memory usually into two partitions: Resident OS and user processes memory address.

- Main memory (physical memory) is a limited resource, must allocate efficiently:
  - Support more processes
  - Fast allocation time
  - Low fragmentation

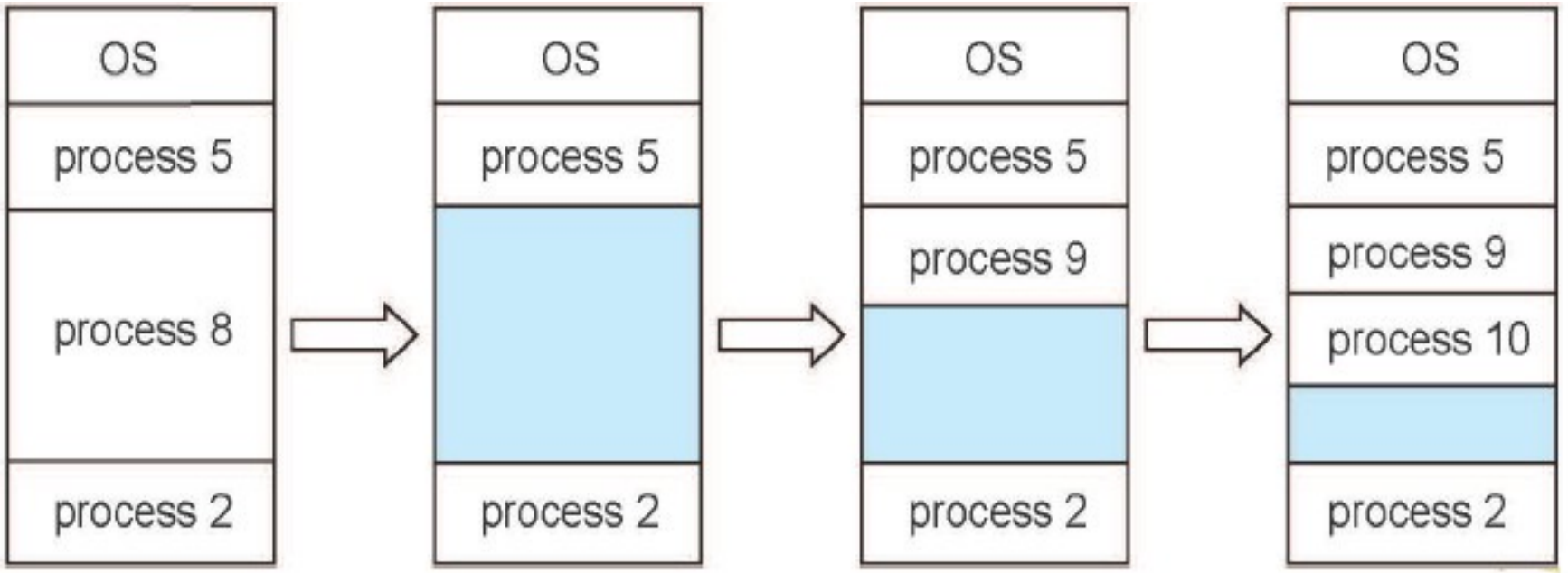
Multiple-Partition Allocation

- Memory is divided into several fixed-sized partitions.
- Each partition may contain exactly one process.
- When a partition is free, a process is loaded into the free partition.



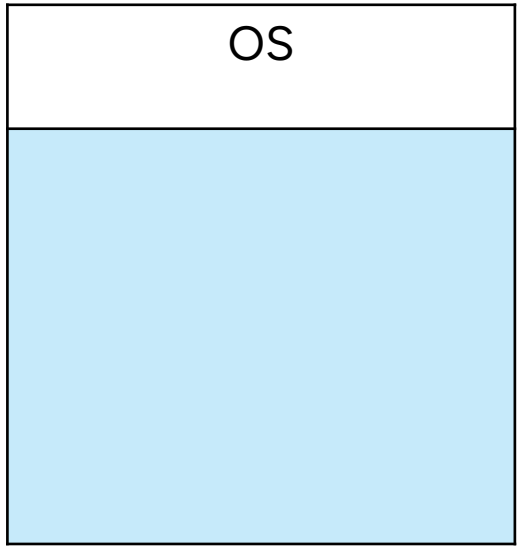
Multiple-Partition Allocation

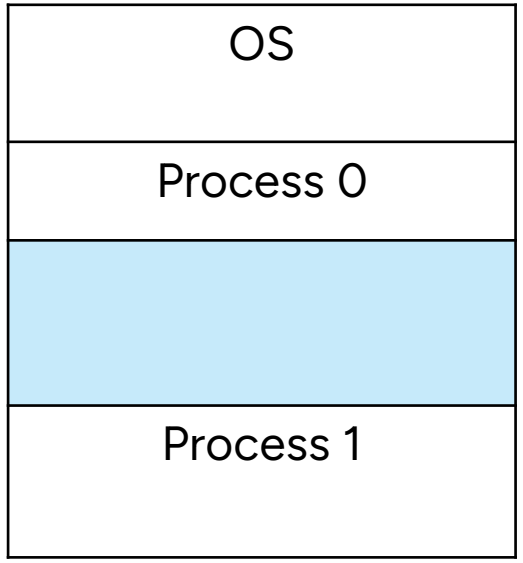
- Variable-partition sizes for efficiency (sized to a given process' needs).
- Hole: block of available memory.
- When a process arrives, it is allocated memory from a hole large enough to accommodate it.
- Process exiting frees its partition, **adjacent free partitions are combined**.

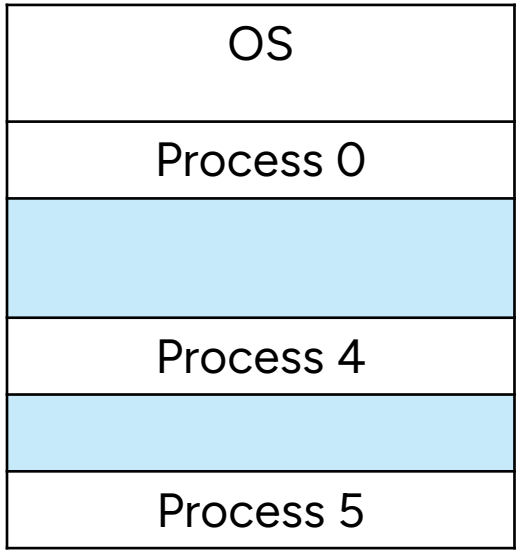


Variable Partition Scheme

- The OS tracks which parts of memory are available or occupied
- Initially, all memory is available, i.e., 1 large block of available memory
- After allocating memory to processes, memory contains holes of variable sizes – variable partition

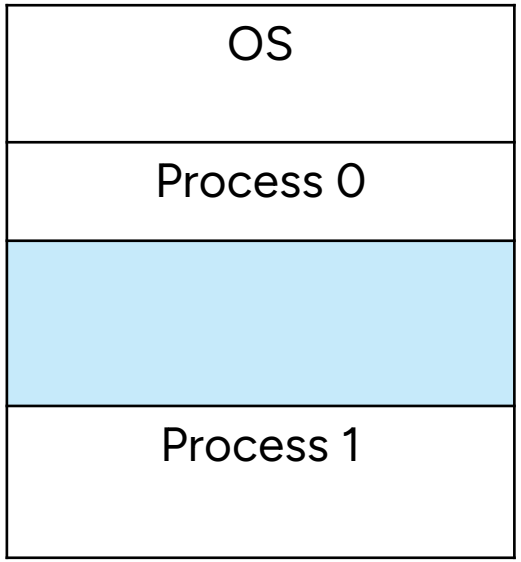


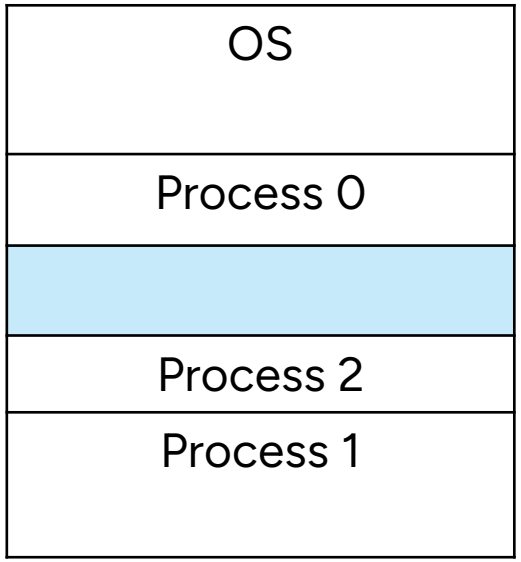




Dynamic Storage-Allocation Problem

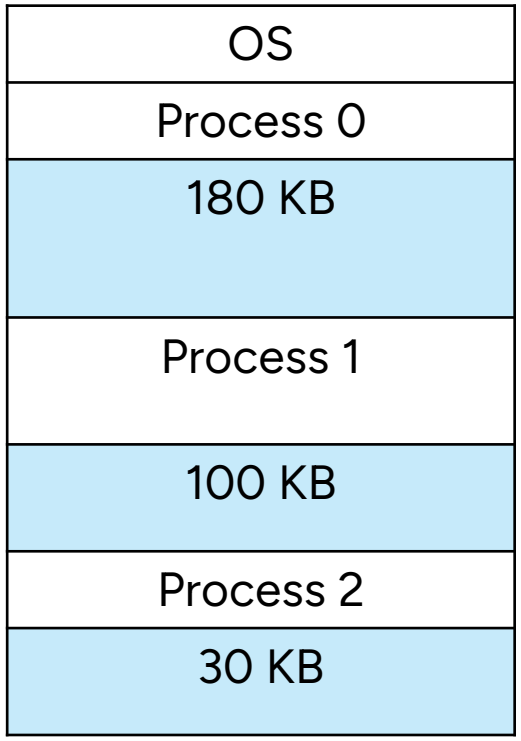
- Available memory blocks is maintained as a list of **holes of various sizes**
- When a process needs memory, need to find a large enough hole for it.
  - Three allocation strategies: first/best/worst fit
  - If the hole is too large, it is split -- one part is allocated to the process; the remaining part is returned
  - When a process terminates, it releases its memory





First Fit Allocation

- Search the list of holes and Allocate the first hole that is large enough
- Complexity?
  - Search can start from beginning or resume from previous search
  - Return as soon as found



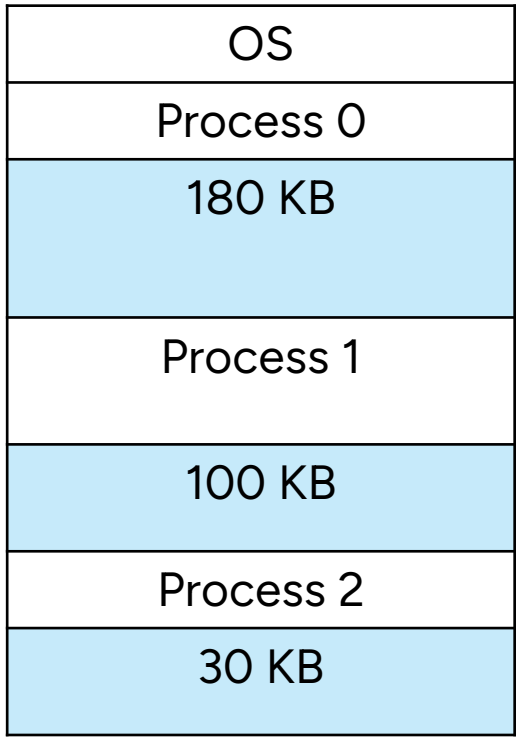
A list of holes in order {180KB, 100KB, 30KB}

A request of 67 KB

A list of holes after allocation {113KB, 100KB, 30KB}

Best Fit Allocation

- Search the list of holes and Allocate the smallest hole that is big enough.
- Complexity?
  - Search needs to iterate all holes in the list unless it is sorted by the size of holes



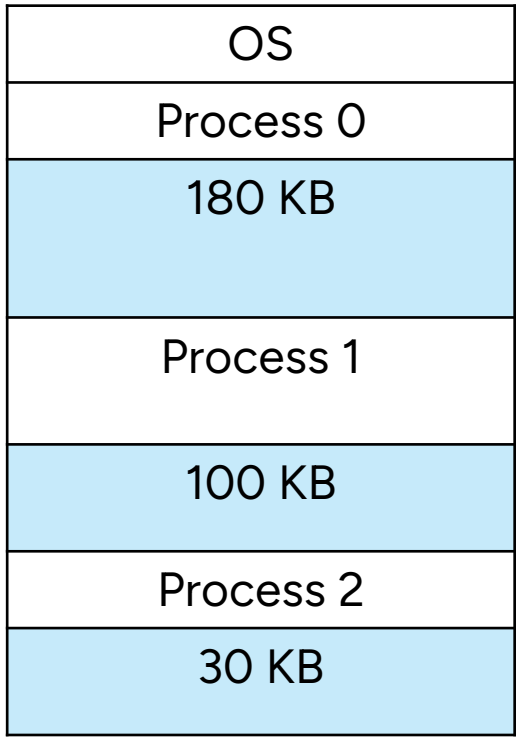
A list of holes in order {180KB, 100KB, 30KB}

A request of 67 KB

A list of holes after allocation {180KB, 33KB, 30KB}

Worst Fit Allocation

- Search the list of holes and Allocate the largest hole.
- Why? So that the remaining part is also large to be usable for future requests



A list of holes in order {180KB, 100KB, 30KB}

A request of 67 KB

A list of holes after allocation {113KB, 100KB, 30KB}



Outline

- Memory Hardware
- Address Binding
- Contiguous Memory Allocation
- **Memory Fragmentation**
- Advanced Memory Topics: Tiered, NUMA

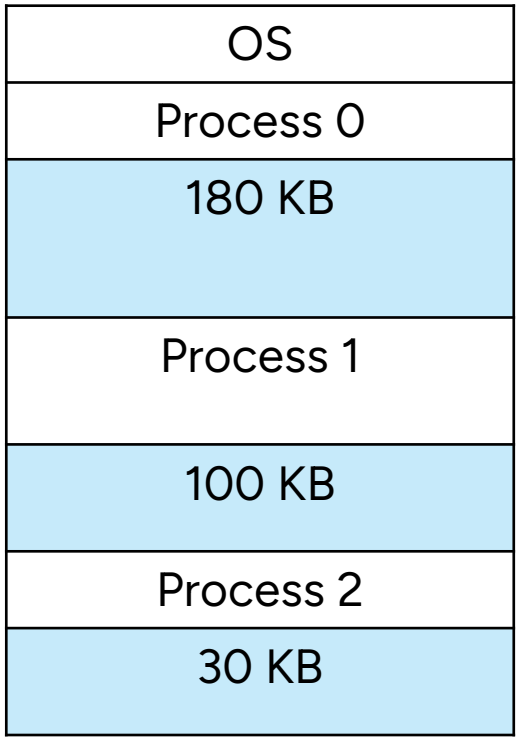


Memory Fragmentation

- Two types:
- External Fragmentation
- Internal Fragmentation

External Fragmentation

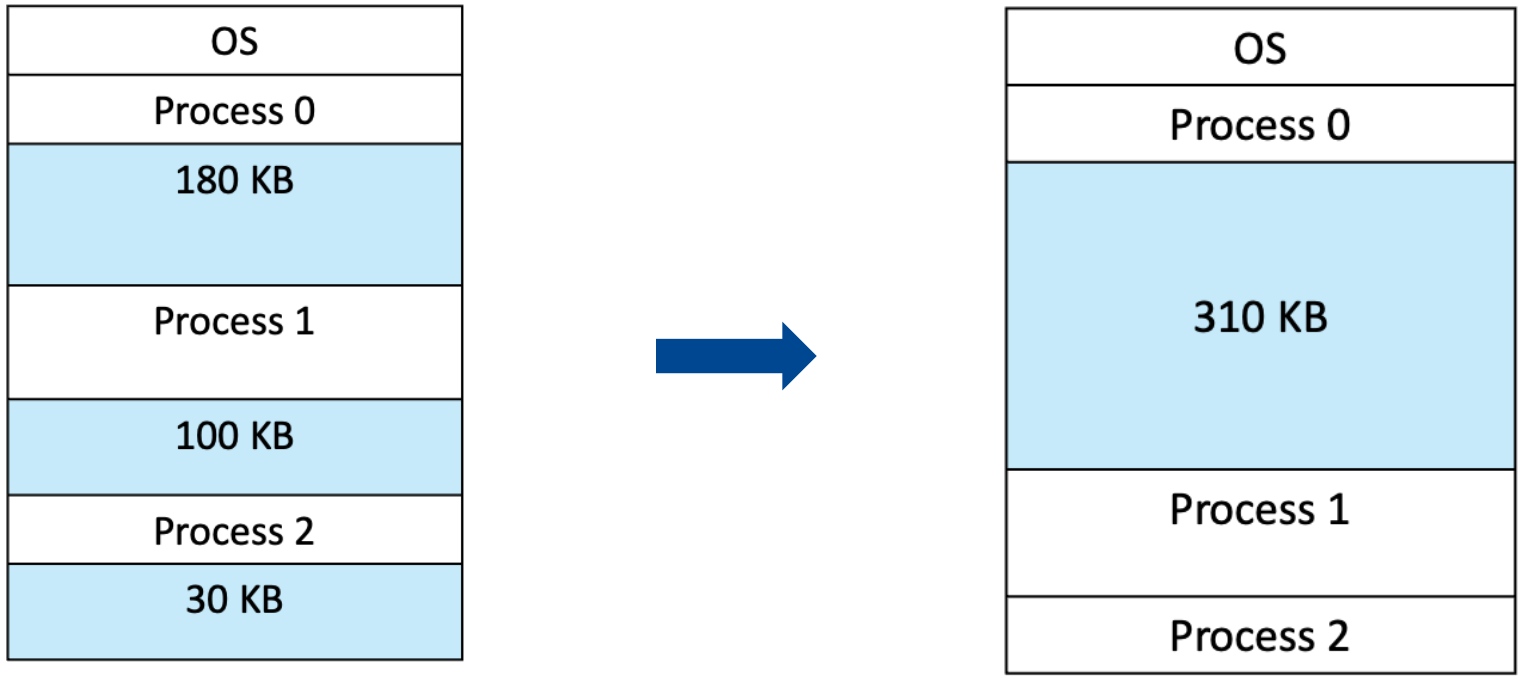
- total memory space exists to satisfy a request, but it is not contiguous.



If requesting an allocation of 280KB?

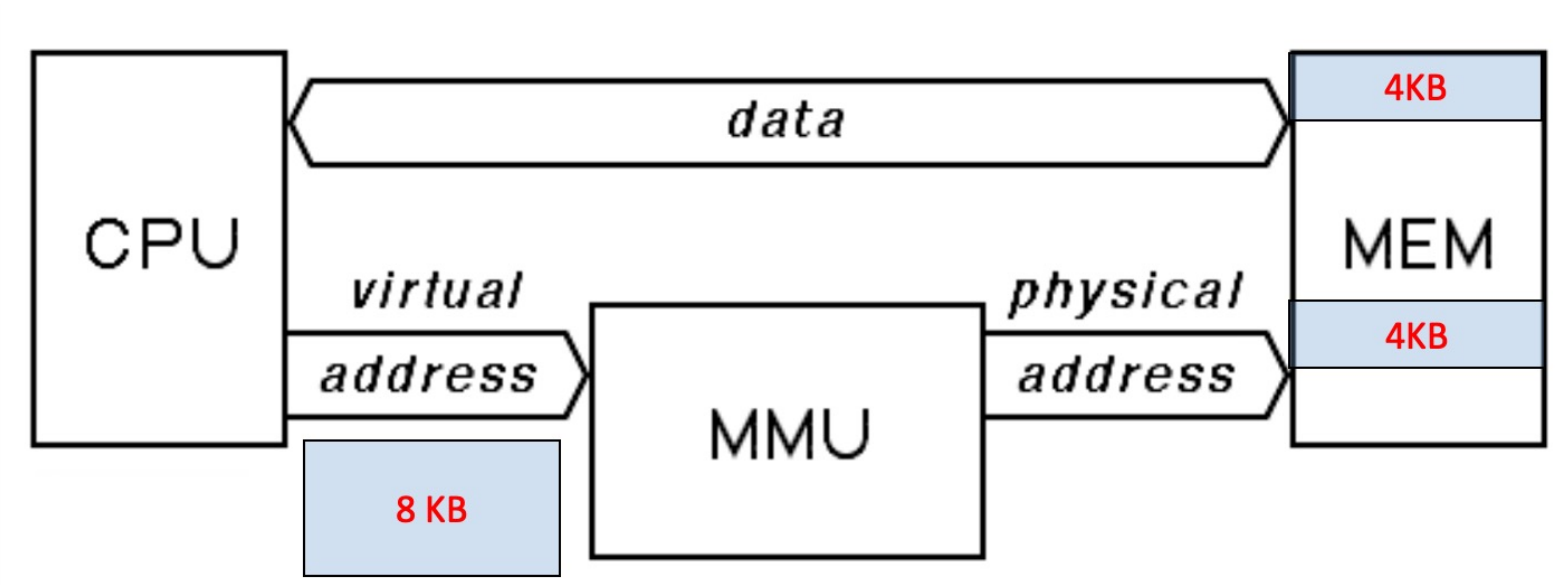
External Fragmentation

- **Compaction**: a solution to the problem of external fragmentation.
- Shuffle memory contents to place all free memory together in one large block.



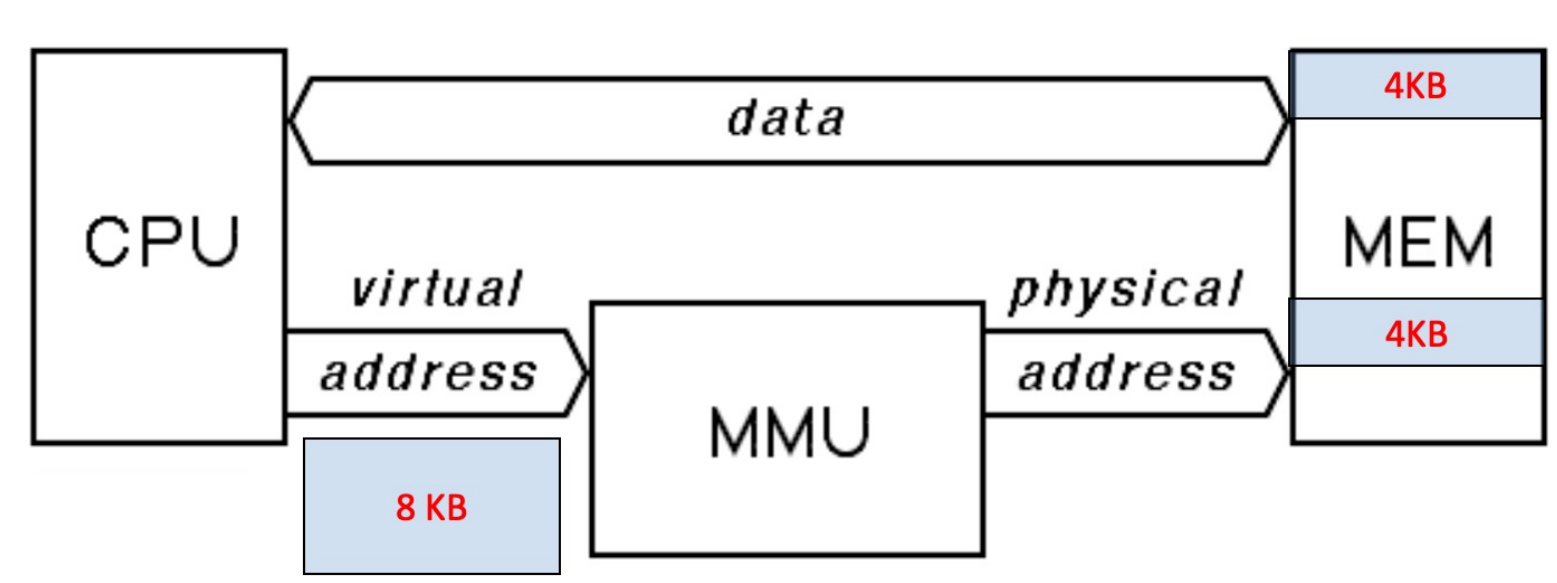
External Fragmentation

- Another solution : permit the logical address space of the processes to be non-contiguous.
- **Paging** (more in Lecture 2.2)
  - In the following example, suppose the page size is 4KB



Internal Fragmentation

- Internal fragmentation: allocated memory may be slightly larger than requested memory
- **Paging** cannot avoid internal fragmentation
- E.g., system page size of 4KB, allocation of 11KB memory, what is the **fragmentation?**



External vs Internal Fragmentation

	External fragmentation	Internal fragmentation
What	free memory exists but is not contiguous scattered in holes	an allocation is larger than requested waste inside the block
Where	between allocations	inside an allocation
Cause	variable-size contiguous allocation	fixed-size units (pages, partitions)
Fix	Compaction, paging	smaller unit size unavoidable with paging

- More in 2.2 Virtual Memory and 2.3 Paging
  - Paging lets any physical frame hold any logical page, so external fragmentation disappears.
  - But **the last page** of a process is rarely full, so paging introduces internal fragmentation



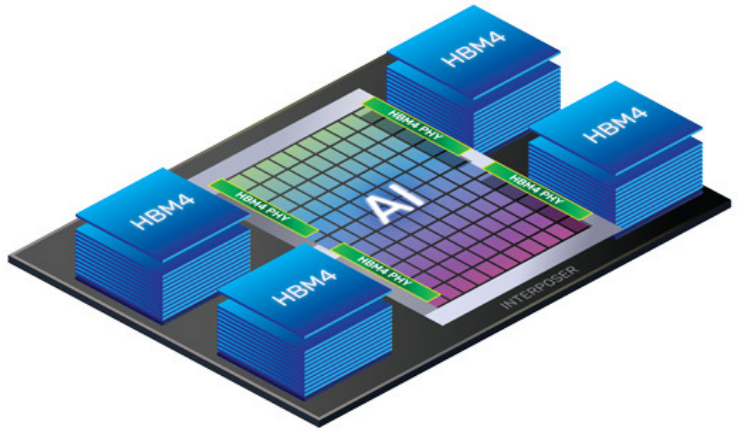
Outline

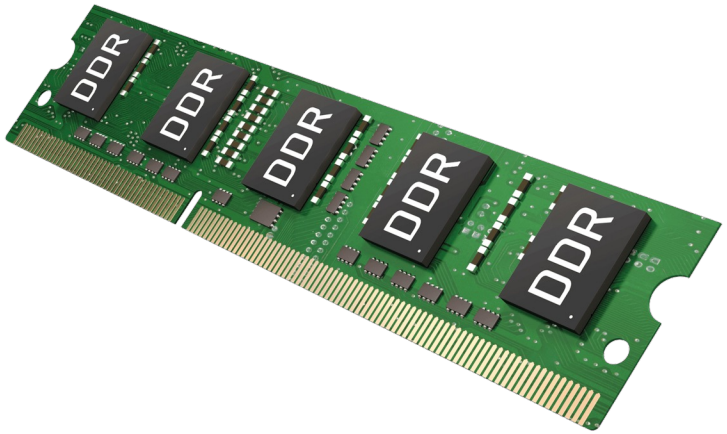
- Memory Hardware
- Address Binding
- Contiguous Memory Allocation
- Memory Fragmentation
- **Advanced Memory Topics: Tiered, NUMA**

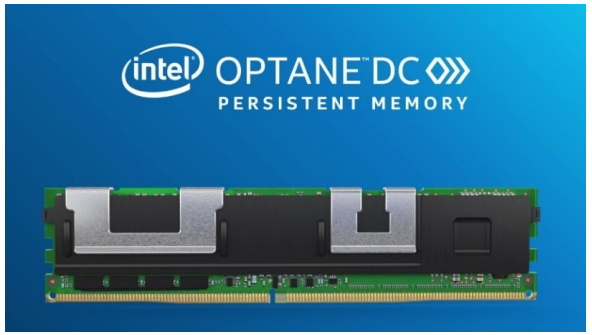
The Memory Hierarchy Is Widening

- For decades, DRAM is the main memory. Now it is one tier among several
- The OS now needs to manage a **tiered** memory system.

Tier	Latency	Peak Bandwidth	Capacity	Persistent?
HBM	~80-100 ns	>TB/s	10s of GB	no
DRAM	~80-100 ns	>100GB/s	100s of GB	no
CXL memory	~200-400 ns	low	TBs	no
Byte-addressable Persistent Memory	~300 ns	medium	TBs	yes







HBM, CXL, and Persistent Memory

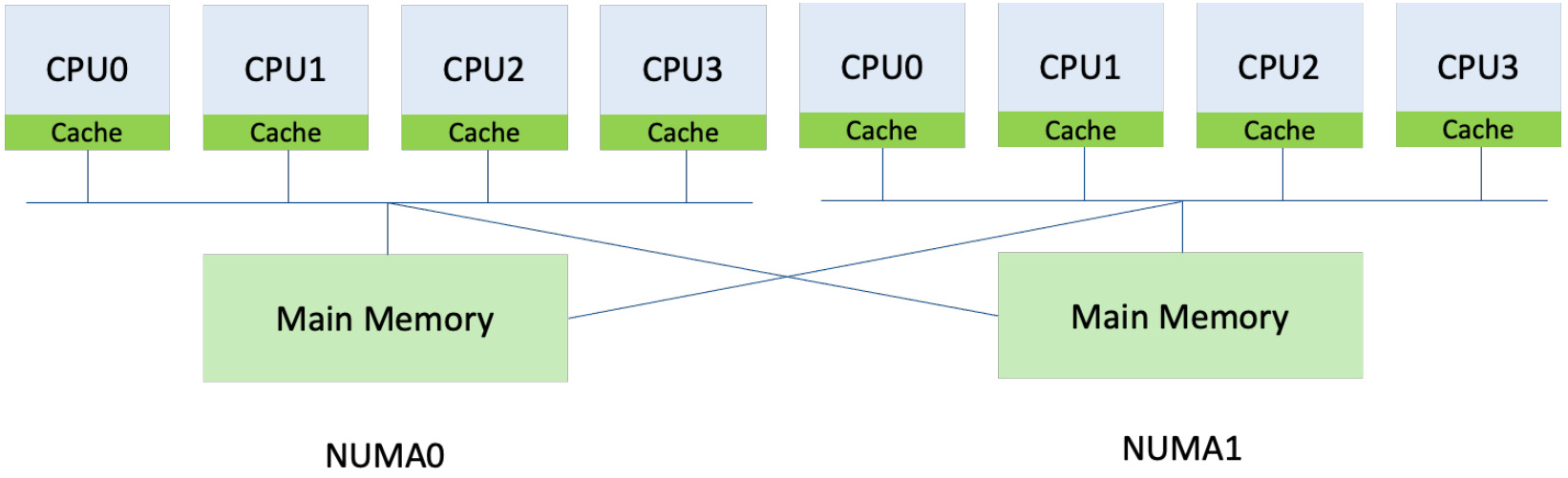
- HBM High-Bandwidth Memory: DRAM dies stacked on-package with a very wide bus.
  - Trades capacity for bandwidth. On GPUs, Intel Xeon Max, and Apple Silicon.
- CXL Compute Express Link: a cache-coherent interconnect over the PCIe physical layer.
  - CXL.mem lets you attach, expand, and pool memory beyond the DIMM slots
  - Memory disaggregation. Higher latency than local DRAM: effectively a NUMA tier.
- Persistent memory: byte-addressable non-volatile memory (NVM)
  - accessed with load/store, not block I/O.
  - Intel Optane pioneered it (discontinued 2022); the tier lives on via CXL-attached persistent memory.

OS-Managed Tiered Memory

- With several tiers, the OS must place hot data in fast memory and cold data in slow.
- **Promotion:** migrate frequently-accessed pages up to DRAM/HBM.
- **Demotion:** migrate cold pages down to CXL/NVM, instead of swapping to disk.
- Linux ranks tiers by an abstract distance and reuses NUMA balancing to promote pages on access.
- The mechanism echoes demand paging and LRU (Lecture 2.3): track access, migrate on the observed pattern.
    - **Demo:** numactl -H: CXL memory or HBM shows up as a **NUMA node without CPU (CPU-less)**.

NUMA: Not All Memory Is Equally Far

- **Non-Uniform Memory Access.** On a multi-socket or chiplet machine, memory splits into **nodes**.
- A CPU reaches its **local** node fast; a **remote** node, across the interconnect, is slower
- So where a page lives, relative to the thread using it, directly affects performance



Placement: First-Touch and numactl

- Default Linux policy: **first-touch.** A page is placed on the NUMA node of the CPU that first writes it
  - not at malloc, but at **first access**. Zero-filled pages are lazy (Lecture 2.2).
- Implication: initialize data on the thread that will use it
- Control with **`numactl`:** --membind, --interleave (spread across multiple NUMA domains for better bandwidth).
- APIs: mbind(), set_mempolicy(), move_pages().
  - Demo: numactl --membind=1 --cpunodebind=1 ./app

AutoNUMA Balancing

- The kernel can migrate pages automatically toward the cores using them: Linux AutoNUMA.
  - It periodically unmaps some pages; the next access triggers a NUMA hint fault that records which node touched the page.
- If a page is consistently accessed remotely, the kernel migrates the page (or moves the thread) to make it local.
- Trade-off:
  - Migration has a cost to pay
  - Good for long-lived, stable access; overhead for short/non-reused access.
  - Toggle: /proc/sys/kernel/numa_balancing.

Back to Our Questions

- **Q1** Address binding has three stages: which are they, and which needs the MMU?
  - Compile time (absolute address; recompile to move), load time (relocatable; fixed when loaded), execution time (movable at runtime, needs the MMU).
- **Q2** Given holes and a request, apply first / best / worst fit and the trade-off?
  - First: fastest. Best: smallest that fits. Worst: poorest.
- **Q3** External vs internal fragmentation: How does paging trade one for the other?